

Cloud Resource Allocation

Using Dynamic Programming — 0/1 Knapsack

An optimal algorithm design approach for maximising workload profit within constrained cloud infrastructure budgets

LAKSH YADAV

1RF24CS094

MIKUL SWAMY

1RF24CS103

MOHAMAD AASHIM

1RF24CS105

MOHAMED ALI

1RF24CS106

The Cloud Resource Challenge

The Scale Problem

Modern cloud providers manage thousands of VMs, containers, and serverless functions simultaneously. Deciding which workloads to schedule — given a fixed resource budget — is non-trivial and directly affects revenue.

Why It Matters

- Resource over-provisioning wastes millions in idle capacity
- Under-provisioning leads to SLA violations and churn
- Greedy heuristics can miss 20–40% of optimal profit
- Exact DP guarantees the globally optimal schedule

30%

avg. idle resource waste
without smart scheduling

$O(n \times W)$

guaranteed optimal
DP time complexity

100%

allocation accuracy vs
greedy approaches

Problem Statement

Given a cloud budget of W resource units and n workload tasks — each requiring r_i units and yielding profit p_i — select a subset S that maximises total profit without exceeding W .

Knapsack Concept	Cloud Equivalent	Example
Knapsack capacity (W)	Total resource budget	10 CPU cores
Item weight (w_i)	Resources required by task	4 cores for ML-Training
Item value (v_i)	Profit / priority score	10 profit units
Selected items	Scheduled workloads	ML-Training, DB-Instance...
0/1 constraint	Each task runs or doesn't	No partial scheduling

Objective: Maximise $\sum p_i \cdot x_i$ subject to $\sum r_i \cdot x_i \leq W$, $x_i \in \{0, 1\}$

Introduction to Dynamic Programming

01

Optimal Substructure

The optimal solution to the full problem contains optimal solutions to sub-problems. For the knapsack, if task i is included optimally, the remaining tasks must also be allocated optimally within the reduced capacity.

02

Overlapping Sub-problems

The same sub-problem (e.g. 'best allocation with 4 units and first 3 tasks') is solved many times in a naive recursion. DP solves each sub-problem once and stores the result in a table — eliminating redundant computation.

03

Memoisation vs Tabulation

Top-down (memoisation): recursive calls cached. Bottom-up (tabulation): iterative table filled in order. We use bottom-up — no stack overhead, cache-friendly, easier to backtrack for the selected set.

The 0/1 Knapsack Algorithm

Recurrence Relation

$dp[i][w]$ = max profit using first i tasks
with capacity w

Base case:

$dp[0][w] = 0$ for all w

Recurrence:

```
if  $r_i > w$ :  
     $dp[i][w] = dp[i-1][w]$   
else:  
     $dp[i][w] = \max(\$   
         $dp[i-1][w],$   
         $dp[i-1][w - r_i] + p_i$   
    )
```

1

Initialise

Create $(n+1) \times (W+1)$ table filled with zeros.

2

Fill Table

For each task i and each capacity w , compute $dp[i][w]$ using the recurrence.

3

Read Answer

$dp[n][W]$ holds the maximum achievable profit.

4

Backtrack

Walk back through the table to recover which tasks were selected.

Time Complexity: $O(n \times W)$ | **Space Complexity:** $O(n \times W) \rightarrow$ reducible to $O(W)$ with 1-D rolling array

Worked Example — Cloud Budget: 10 CPU Units

Input Tasks

Task	CPU Units	Profit
Web-Server	2	6
ML-Training	4	10
DB-Instance	3	7
Cache-Node	1	3
Batch-Job	5	12
API-Gateway	2	5

Optimal Allocation Result

ML-Training	4 cores	+10
DB-Instance	3 cores	+7
Cache-Node	1 cores	+3
Web-Server	2 cores	+6
TOTAL	10 / 10	26

Batch-Job (5 cores, profit 12) is excluded — including it would drop profit to 22. DP guarantees the globally optimal choice.

DP Table — Filling Process (capacity $W = 6$ for clarity)

Each cell $dp[i][w]$ stores the maximum profit achievable using the first i tasks with capacity w . Highlighted cells show where a task was included (value improved over row above).

Task \ Cap	w=0	w=1	w=2	w=3	w=4	w=5	w=6
i=0 (none)	0	0	0	0	0	0	0
i=1 Web(2,6)	0	0	6	6	6	6	6
i=2 ML(4,10)	0	0	6	6	10	10	16
i=3 DB(3,7)	0	0	6	7	10	13	16
i=4 Cache(1,3)	0	3	6	9	10	13	16

Backtrack: $dp[4][6]=16 \rightarrow$ Cache included ($dp[4][6] \neq dp[3][6]$) $\rightarrow dp[3][5]=13 \rightarrow$ DB included $\rightarrow dp[2][2]=6 \rightarrow$ ML excluded ($4 > 2$) $\rightarrow$ Web included

Implementation Details

Python — Core DP (src/knapsack.py)

```
@dataclass
class Task:
    name: str
    resource: int
    profit: int

def allocate(tasks, capacity):
    n = len(tasks)
    dp = [[0]*(capacity+1)
          for _ in range(n+1)]

    for i in range(1, n+1):
        task = tasks[i-1]
        for w in range(capacity+1):
            dp[i][w] = dp[i-1][w]
            if task.resource <= w:
                val = dp[i-1][w-task.resource]
                    + task.profit
                if val > dp[i][w]:
                    dp[i][w] = val

    return dp[n][capacity], selected
```

Task Data Model

Dataclass with name, resource units required, and profit — clean mapping of the knapsack item abstraction.

Bottom-Up Fill

$O(n \times W)$ nested loops. Each cell computed once. No recursion overhead; cache-friendly row-by-row traversal.

Backtracking

Walk from $dp[n][W]$ upward. If $dp[i][w] \neq dp[i-1][w]$ then task i was included; decrement capacity accordingly.

CLI Interface

main.py accepts custom tasks.json or runs a built-in 6-task demo. Prints the full DP table for inspection.

System Architecture

Input Layer

JSON task definitions (name, resource, profit) or built-in demo. Supports custom cloud workload scenarios via tasks.json.

Core DP Engine

knapsack.py — bottom-up 2-D DP table builder + backtracking algorithm. Deterministic $O(n \times W)$ time, $O(n \times W)$ space.

CLI Interface

main.py — CLI entry point. Prints input tasks, full DP table, optimal selection, resource utilization, and max profit.

Web Frontend

docs/index.html + app.js — browser-based interactive visualiser (GitHub Pages). Enter tasks, see DP table and result live.

Benchmark Suite

benchmark.py — empirical runtime curves varying n (tasks) and W (capacity). Validates $O(n \times W)$ growth experimentally.

Complexity Analysis

Time Complexity — $O(n \times W)$

n tasks	W capacity	DP Cells	Approx. Runtime
10	10	110	< 1 μ s
100	1,000	100,100	~1 ms
1,000	10,000	10 M	~10 s
10,000	100,000	1 B	hours (impractical)

Space Complexity — $O(n \times W) \rightarrow O(W)$

Approach	Space	Trade-off
2-D table	$O(n \times W)$	Full table; backtrack possible
1-D rolling	$O(W)$	Minimal space; can't backtrack
Space-optimised	$O(W)$	Use bitmask for selection recovery
Branch-Bound	$O(n)$	Prunes tree; faster in practice

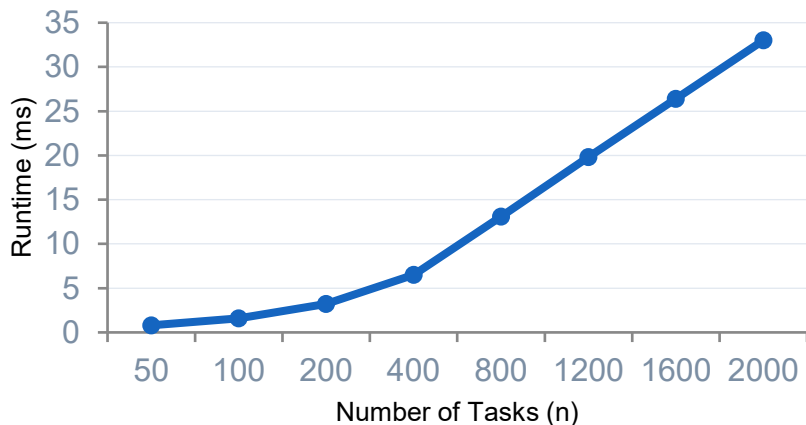
Important: Pseudo-Polynomial Complexity

0/1 Knapsack DP is $O(n \times W)$ — polynomial in n but exponential in the number of bits encoding W (i.e. $\log W$). For large W , consider FPTAS (Fully Polynomial-Time Approximation Scheme) which achieves $(1-\epsilon)$ optimal in $O(n^2/\epsilon)$ time, or Branch-and-Bound which prunes the search tree and is fast in practice for sparse high-profit solutions.

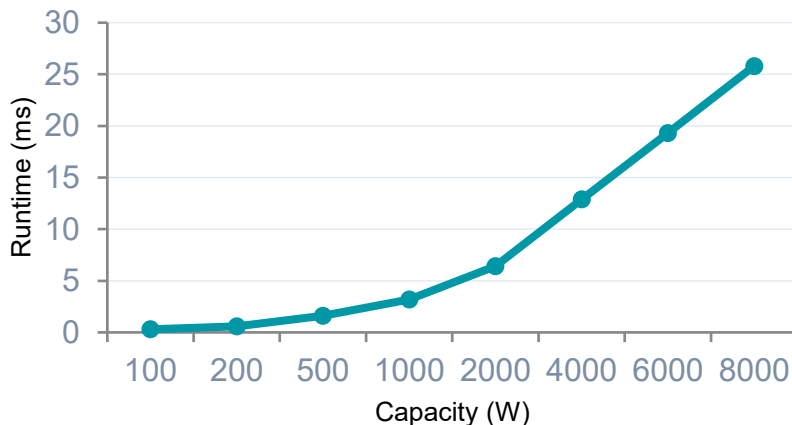
Performance Benchmarks

Empirical runtime measured on Python 3.13 / Intel Core i5 / 8 GB RAM. Tasks generated with fixed seed for reproducibility.

Vary n ($W = 500$ fixed)



Vary W ($n = 200$ fixed)



Linear

~33 ms

< 1 ms

growth in both n and W — confirms $O(n \times W)$

for $n=2000$, $W=500$ on standard hardware

for typical cloud tasks ($n < 100$, $W < 500$)

Why Not Greedy? — DP vs Greedy Comparison

A greedy algorithm (sort by profit/resource ratio, pick highest first) is fast but does NOT guarantee the optimal solution for the 0/1 Knapsack problem.

Criterion	Greedy (Ratio)	DP (0/1 Knapsack)
Optimality	Not guaranteed — can miss global optimum	Always optimal
Time Complexity	$O(n \log n)$ — sort + scan	$O(n \times W)$
Space	$O(1)$	$O(n \times W)$ or $O(W)$
Partial tasks	Works well for fractional knapsack	Required for 0/1 constraint
Cloud suitability	May leave resource unused sub-optimally	Maximises revenue within budget
Counter-example	Items $\{(3,4),(3,4),(5,7)\}$ $W=6 \rightarrow$ greedy=7	DP $\rightarrow 8$ (pick both (3,4) items)

Conclusion: DP is the correct choice for 0/1 cloud resource allocation — greedy works only when tasks are fractionally divisible.

Conclusion

01

Exact Optimality

DP guarantees the maximum-profit allocation — no greedy shortcut can match this for the 0/1 constraint.

02

Polynomial Tractable

$O(n \times W)$ runtime is practical for real cloud scenarios (hundreds of tasks, thousands of resource units).

03

Full Traceability

The DP table provides a complete audit trail — every allocation decision can be explained and reproduced.

04

Extensible

The model extends to multi-dimensional knapsack (CPU + RAM + bandwidth) and multi-period scheduling.

Thank You

References

[1]

Cormen, Leiserson, Rivest, Stein — Introduction to Algorithms, 3rd Ed.

Chapter 15: Dynamic Programming. MIT Press, 2009. Foundational reference for the DP framework and correctness proofs.

[2]

Kellerer, Pferschy, Pisinger — Knapsack Problems

Springer, 2004. Comprehensive treatment of all knapsack variants, approximation schemes, and exact algorithms.

[3]

Buyya, Broberg, Goscinski — Cloud Computing: Principles and Paradigms

Wiley, 2011. Background on cloud resource management models and SLA-driven scheduling.

[4]

Project Repository — github.com/Mohamed-Ali-4/cloud-resource-allocation-dp

Full source code, benchmark scripts, web frontend, and ADA report for this implementation.